

Practice Assignment 5

LIBRARY MANAGEMENT SYSTEM

Following Practice Assignment 4, further develop the new model in the .Net Core framework.

Authentication & Authorization (JWT)

Learning Objectives

By the end of this lab, you should be able to:

- Authenticate users using credentials and issue JWTs.
- Protect API routes with [Authorize].
- Implement role-based authorization (e.g., Admin, User).
- Understand how claims and tokens work in stateless APIs.

Required Setup

1. Step 1 – Add JWT Settings to appsettings.json (MUST BE FIRST)

```
"JwtSettings": {  
  "SecretKey": "your-strong-random-key-should-be-32+chars",  
  "Issuer": "YourCompany.AuthServer",  
  "Audience": "YourCompany.ClientApp",  
  "ExpiryMinutes": 60  
}
```

- ☐ This configuration must be added **before** configuring JWT in Program.cs.

2. Step 2 – Install JWT Authentication NuGet Package

In your NuGet package manager console:

Microsoft.AspNetCore.Authentication.JwtBearer

Ensure the package version matches your .NET Core project version.

3. Step 3 – Configure JWT in Program.cs

```
var builder = WebApplication.CreateBuilder(args);
// Load JWT settings from configuration
var jwtSettings = builder.Configuration.GetSection("JwtSettings");
var key = jwtSettings["SecretKey"];
var issuer = jwtSettings["Issuer"];
var audience = jwtSettings["Audience"];
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = issuer,
            ValidAudience = audience,
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(key))
        };
    });
builder.Services.AddAuthorization();
var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
```

Exercise 1: Create the User Model and Load from Database

The **User** entity was already created in the previous exercise. In this exercise, you only need to create the **Role** entity. However, if you don't have both yet, create both the **User** and **Role** entities and retrieve users from the database using **AppDbContext**.

Example:

// Use the class User from the previous exercise (Lab 4) for this model

```
public class User
{
    .....
    public string Password { get; set; } // In real systems, this must be hashed
    .....
}
```

Example:

```
Public class Role
{
    private int RoleId {get; set;};
    private string RoleName {get; set;}; // e.g., "Admin", "User", "Librarian" ,....
    private string Description {get; set;};
    public ICollection<UserRole> UserRoles {get; set;};
}
```

Exercise 2: Implement Login with JWT in AuthController
DTO – LoginRequest.cs

```
public class LoginRequest
```

```
{
    public string Username { get; set; }
    public string Password { get; set; }
}
```

Controller – AuthController.cs

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class AuthController : ControllerBase
```

```
{
    private readonly AppDbContext _context;
    private readonly IConfiguration _configuration;

    public AuthController(AppDbContext context, IConfiguration configuration)
    {
        _context = context;
        _configuration = configuration;
    }
}
```

```
[HttpPost("login")]
```

```
public IActionResult Login([FromBody] LoginRequest request)
```

```
{
    var user = _context.Users.FirstOrDefault(u =>
        u.Username == request.Username && u.Password ==
request.Password);
    if (user == null)
```

```

        return Unauthorized();
        var token = GenerateJwtToken(user);
        return Ok(new { token });
    }
    private string GenerateJwtToken(User user)
    {
        var claims = new[]
        {
            new Claim(ClaimTypes.Name, user.Username),
            new Claim(ClaimTypes.Role, user.Role)
        };
        var key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["JwtSettings:SecretKey"]));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _configuration["JwtSettings:Issuer"],
            audience: _configuration["JwtSettings:Audience"],
            claims: claims,
            expires: DateTime.Now.AddMinutes(60),
            signingCredentials: creds
        );
        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}

```

Exercise 3: Create a Protected Endpoint

Endpoint: GET /api/users/me

[ApiController]

[Route("api/[controller]")]

public class UsersController : ControllerBase

```

{
    [HttpGet("me")]
    [Authorize]
    public IActionResult GetMe()
    {
        var username = User.Identity?.Name;
        var role = User.FindFirst(ClaimTypes.Role)?.Value;
        return Ok(new { username, role });
    }
}

```

//This endpoint requires a valid JWT in the Authorization header.

Exercise 4: Role-Based Authorization

Endpoint: GET /api/admin/dashboard

```
[HttpGet("dashboard")]
[Authorize(Roles = "Admin")]
public IActionResult AdminDashboard()
{
    return Ok("Welcome, Admin!");
}
```

Users without the **Admin** role should receive **403 Forbidden**.

Exercise 5: Test with Swagger / Postman

1. Test login with valid and invalid credentials:
 - o POST /api/auth/login
2. Copy the returned JWT token.
3. Use the token in:
 - o GET /api/users/me (requires any valid user)
 - o GET /api/admin/dashboard (requires Admin role)

Set the Authorization header in format: Authorization: Bearer <your_token_here>

Submission Checklist

Make sure the following are included in your submission:

- POST /api/auth/login returns a valid JWT on success.
- GET /api/users/me requires authentication via [Authorize].
- GET /api/admin/dashboard requires Admin role via [Authorize(Roles = "Admin")].
- JWT secret and settings stored in appsettings.json.
- Clean and testable code, verified via Swagger or Postman.
- User data retrieved from the database (not hardcoded list).

Exercise 6 – Implement Policy-Based Authorization

Create a custom authorization policy named "**ActiveUserOnly**" that allows access only to users whose account is active (**IsActive == true**).

Apply this policy to an endpoint that returns a list of active users.

Exercise 7 – Create a Role + Claim Combination Policy

Create a policy called "**AdminOrLibrarian**" that grants access if the user has either the "**Admin**" or "**Librarian**" role.

Apply this policy to the **/api/reports/top-borrowed** endpoint.

Exercise 8 – Implement a Minimum Membership Duration Policy

Define a policy named `"MinimumMembership"` that allows only users who have been registered for more than **30 days**.

Apply this policy to the endpoint `/api/loans/history` so that only long-term users can view their loan history.

Exercise 9 – Implement Policy for Active Category Management

Create a policy `"ManageActiveCategories"` that only allows access to users with the `"Admin"` role and the claim `"CanManageCategories" = "True"`.

Use this for the `/api/categories` management endpoints.

Exercise 10 – Create Policy for Verified Email Users

Implement a policy called `"VerifiedEmailOnly"` that restricts access to certain features (like uploading PDFs or avatars) to users with verified emails (`EmailConfirmed = true`).

Apply it to `/api/books/upload`.